# GETTING STARTED WITH LANGGRAPH

Stateful, Multi-Step AI Agents — Quick Reference

O'Reilly Live Training | Lucas Soares

## AT A GLANCE: 4 CORE BUILDING BLOCKS

**State**
Shared data, a TypedDict

**Node**
Function: state → state

**Edge**
Fixed or conditional link

**Graph**
Compiled, runnable app

## 1. BUILDING YOUR FIRST GRAPH

### The Pattern

- Define `State` as a `TypedDict` — the shared data every node reads and updates
- Write nodes as plain functions: take state in, return a partial update
- `add_node` registers a node; `add_edge` wires fixed transitions
- `START` / `END` are the graph's fixed entry/exit points
- `compile()` turns the builder into a runnable app

**notebooks/1.0-langgraph-fundamentals.ipynb**

```
from langgraph.graph import StateGraph,
START, END
from typing_extensions import TypedDict

class State(TypedDict):
    graph_state: str

def node1(state):
    state["graph_state"] = "node 1"
    return state

builder = StateGraph(State)
builder.add_node("node1", node1)
builder.add_edge(START, "node1")
builder.add_edge("node1", END)
app = builder.compile()

app.invoke({"graph_state": ""})
```

## 2. STATE & REDUCERS — THE CLASSIC FOOTGUN

### Override vs. Accumulate

- A plain `list` field is **overridden** on every node update — later writes silently erase earlier ones
- `Annotated[list, add_messages]` makes the field **accumulate** instead
- `MessagesState` is a ready-made state with `messages` already wired to `add_messages`
- Two nodes in the same parallel step re-emitting the *same* unchanged field can still trigger an `InvalidUpdateError` — give every field a reducer

Chat History · Parallel Fan-Out

**Reducer Fix**

```
from typing import Annotated
from langgraph.graph.message import
add_messages

class State(TypedDict):
    # Annotated + add_messages => append,
    # not overwrite, on every node return
    messages: Annotated[list,
add_messages]

def my_node(state):
    return {"messages":
[HumanMessage("hi")]}
```

## 3. REACT AGENT: TOOLS + CONDITIONAL ROUTING

### What It Does

- `llm.bind_tools(tools)` lets the model decide when to call a tool
- `ToolNode([...])` executes whatever tool call the LLM emitted
- `tools_condition` is a prebuilt conditional edge: routes to the tool node if a tool call is present, else to `END`
- This loop (assistant → tools → assistant) is the core of nearly every agent

Web Search · Function Calling

**notebooks/2.0-react-agent-with-memory.ipynb**

```
from langgraph.graph import MessagesState,
StateGraph, START
from langgraph.prebuilt import
tools_condition, ToolNode

def assistant(state: MessagesState):
    return {"messages":
[llm_with_tools.invoke(state["messages"])]
}

builder = StateGraph(MessagesState)
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "assistant")
builder.add_conditional_edges("assistant",
tools_condition)
builder.add_edge("tools", "assistant")
```

## 4. MEMORY: CHECKPOINTERS & THREADS

### What It Does

- A checkpointer (e.g. `InMemorySaver`) persists graph state after every step
- Pass `checkpointer=memory` to `compile()` to turn it on
- Every call needs a `thread_id` in `config` — that's what isolates one conversation from another
- Same `thread_id` across calls → the graph resumes with full history

Multi-Turn Chat · Session Isolation

**Usage Example**

```
from langgraph.checkpoint.memory import
InMemorySaver

memory = InMemorySaver()
graph =
builder.compile(checkpointer=memory)

config = {"configurable": {"thread_id":
"1"}}
graph.invoke({"messages": [("user",
"hi")]}, config)

# Same thread_id => remembers prior turns
graph.invoke({"messages": [("user", "and
then?")]}, config)
```

## COURSE NOTEBOOKS

| # | Notebook | Core Pattern |
|---|---|---|
| 1.0 | LangGraph Fundamentals | State, nodes, edges, conditional routing, reducers |
| 2.0 | ReAct Agent with Memory | `tools_condition`, `ToolNode`, checkpointer, thread isolation |
| 3.0 | RAG Agent | Vectorstore retrieval, document grading, web-search fallback routing |
| 4.0 | Research Report Agent | Multi-node graph, parallel fan-out, conditional loops, structured output |

## KEY API REFERENCE

| Call | What It Does |
|---|---|
| `StateGraph(State)` | Start a new graph builder typed on your state schema |
| `add_node(name, fn)` | Register a node function under a name |
| `add_edge(a, b)` | Fixed transition from node `a` to node `b` |
| `add_conditional_edges(n, fn)` | Route dynamically from node `n` based on `fn(state)`'s return value |
| `set_conditional_entry_point(fn)` | Pick the graph's first node dynamically instead of a fixed `START` edge |
| `compile(checkpointer=...)` | Build the runnable graph, optionally with persistence |
| `.invoke(state, config)` | Run the graph to completion, return final state |
| `.stream(state, config)` | Run the graph, yielding state after each node/step |

### AGENTIC DESIGN PATTERNS (FROM THE SLIDE DECK)

- **Plan & Execute** — a planner node lays out steps up front; executor nodes carry them out
- **Supervisor** — a router node delegates each turn to the right specialized sub-agent
- **Multi-Agent** — specialized agents collaborate, each owning part of the task
- **Flow Engineering** — explicit, code-defined control flow instead of leaving control fully to the LLM

**Pro Tip**

Give every state field a reducer (`add_messages` or your own) once more than one node can write to it — the default "last write wins" behavior is rarely what you want in a graph with branches.

**⚠ Model Gotcha**

Tool-calling with a reasoning model may need an explicit setting to work reliably — e.g. this course pins `reasoning_effort="none"` when binding tools, since higher reasoning effort can suppress tool calls.